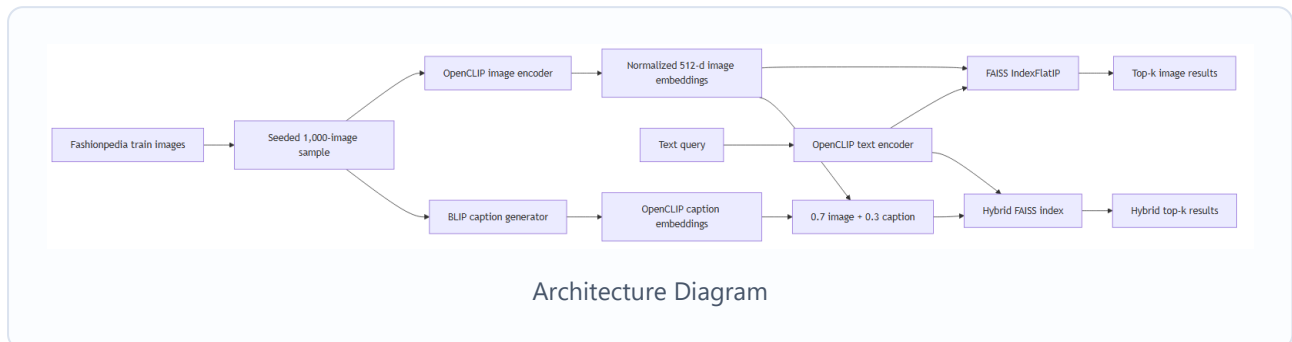


Fashion Retrieval Report



Problem Statement

The goal is to retrieve fashion images using natural language descriptions. A user should be able to enter a query such as "Someone wearing a blue shirt sitting on a park bench" and receive visually relevant images from a fashion image collection.

Dataset

The notebook uses the Kaggle dataset `tpgd01/fashionpedia`. The downloaded archive is unzipped into `fashionpedia`, and images are read from:

```
/content/fashionpedia/final_dataset/final_dataset/train
```

Notebook output shows `45,623` train images and `1,158` test images. The implemented experiment samples `1,000` train images with `random.seed(42)`.

Alternative Approaches

Possible approaches include:

- Metadata-only search using labels or manually written tags.
- Supervised classification followed by filtering on predicted categories.
- Text-only caption search using generated captions and sentence embeddings.
- Image-text contrastive retrieval using CLIP-style embeddings.
- Multistage retrieval with vector search followed by reranking.

Metadata-only and classifier-only approaches are limited when user queries are descriptive or compositional. Caption-only retrieval can help with semantics but may miss visual details. The notebook therefore uses OpenCLIP as the core shared image-text embedding model and BLIP captions as an auxiliary semantic signal.

Chosen Architecture

The implemented architecture has two retrieval paths:

1. OpenCLIP retrieval: - Encode images with OpenCLIP. - Normalize embeddings. - Store embeddings in FAISS `IndexFlatIP`. - Encode the text query with OpenCLIP. - Search FAISS for the top-k image embeddings.
2. Hybrid retrieval: - Generate BLIP captions for each image. - Encode captions with OpenCLIP text encoding. - Combine image and caption embeddings using $0.7 * \text{image_embedding} + 0.3 * \text{caption_embedding}$. - Normalize the hybrid embeddings. - Search a second FAISS index using the OpenCLIP text query embedding.

Why OpenCLIP

OpenCLIP is a strong fit because it maps images and natural language into a shared embedding space. This enables zero-shot retrieval without training a task-specific classifier on the Fashionpedia sample. The notebook uses `ViT-B-32` with `laion2b_s34b_b79k` pretrained weights and produces 512-dimensional embeddings.

Why BLIP

BLIP adds a captioning signal that can describe image contents in natural language. This helps expose semantic details that may be easier to match through text, such as clothing type, pose, or context. In the notebook, BLIP captions are generated with `Salesforce/blip-image-captioning-base` and then embedded with the same OpenCLIP text encoder.

Hybrid Retrieval

The hybrid representation combines visual and caption-based semantics:

```
hybrid_embedding = 0.7 * image_embedding + 0.3 * caption_embedding
```

The result is normalized before indexing. This preserves the image embedding as the dominant signal while adding caption information as a secondary signal.

FAISS Indexing

The notebook uses FAISS `IndexFlatIP`. Because embeddings are normalized, inner product search behaves like cosine similarity. `IndexFlatIP` is exact and simple, making it appropriate for a 1,000-image assignment-scale experiment.

Evaluation

The notebook demonstrates qualitative evaluation through example natural language queries and plotted top-k retrieval grids. It does not include labeled relevance judgments, recall, precision, nDCG, or other quantitative metrics. The sample result images in this repository are extracted directly from the notebook outputs.

Trade-offs

- `IndexFlatIP` is exact but scans all vectors, so it is not the most scalable option for very large datasets.
- BLIP caption generation adds semantic value but increases preprocessing time.
- The hybrid weight `0.7 / 0.3` is fixed in the notebook and not tuned against a validation set.
- Zero-shot retrieval avoids task-specific training but may miss fine-grained fashion attributes.
- Caption quality depends on the BLIP model and may include incomplete or noisy descriptions.

Scalability to 1 Million Images

For 1 million images with 512-dimensional `float32` embeddings, raw embedding storage is approximately:

```
1,000,000 * 512 * 4 bytes = about 2 GB
```

An exact `IndexFlatIP` search over 1 million vectors may be too slow for interactive use on CPU. A production-scale system should consider:

- FAISS IVF or HNSW indexes for approximate nearest-neighbor search.
- Product quantization to reduce memory usage.
- GPU FAISS for faster batch indexing and querying.
- Offline embedding and caption generation jobs.
- Persistent metadata storage for image paths, captions, and dataset splits.
- Periodic index rebuilds or incremental index updates depending on ingestion needs.

Zero-Shot Retrieval

The system is zero-shot because it relies on pretrained OpenCLIP and BLIP models rather than training on task-specific query-image pairs. This is useful for internship-scale prototyping because it supports flexible natural language queries immediately after embedding the image collection.

Future Work

- Add labeled evaluation queries and compute retrieval metrics.
- Tune hybrid weights with a validation set.
- Compare OpenCLIP backbones and pretrained checkpoints.
- Add reranking with a cross-modal model for the top retrieved candidates.
- Support approximate FAISS indexes for larger datasets.
- Package the workflow as a CLI or lightweight demo app.
- Add tests for embedding shape, FAISS index persistence, and image-path alignment.